

Physics-Informed U-Net: SE232 Final Project

Jeremy Doan

1. Introduction

Partial differential equations (PDE) are prevalent in many engineering problems, such as heat and elasticity. In many cases, a closed-form solution is not easily obtainable. As a result, several methods have been developed for approximating the solution to these kinds of problems. One such method is the Finite Element Method (FEM). FEM is a numerical method used to approximate the solution of differential equations relevant to engineering by discretizing the problem domain into simple elements [1]. Some problems that can be solved using FEM are heat conduction and elasticity problems. Another discrete method is the Reproducing Kernel Particle Method (RKPM), which discretizes the domain into a fixed point cloud instead of a mesh [2–4]. RKPM provides an alternative to FEM that does not require a mesh and is very effective for solving large deformation problems and for modeling the evolution of damage and fractures in materials and structures. Regardless of the discretization method, computation time can be costly, depending on the complexity of the problem. Therefore, one possible solution is to leverage machine learning techniques to learn the mapping between loading conditions and displacements, in the case of an elasticity problem.

Recently, deep learning approaches have been adopted for learning complex mappings in data. Convolutional Neural Networks (CNN) are a type of deep learning model that has been specialized for spatially continuous data. For example, U-Net was developed for semantic segmentation of biomedical images [5]. The model is based on a symmetric encoder-decoder structure, which are used to extract both global and local features [6]. Moreover, U-Net has proven useful in general segmentation and image processing tasks. Since the U-Net is advantageous in pixel-wise reconstruction, there is potential for application to PDE-based problems. Deep learning networks have been coupled with RKPM to solve PDE problems [7, 8]. Baek et al. demonstrated the effectiveness of CNN-based deep networks in modeling sharp transitions in discontinuous solutions. This was used to enrich standalone RKPM shape functions in solving elasticity problems.

A realistic displacement field obtained as the solution to an elasticity problem is spatially continuous unless a constraint is imposed. Therefore, it may be possible for a U-Net to capture the distribution of displacements as it would a medical anomaly in an X-ray scan. In this study, a U-Net model was trained on a dataset obtained by an RKPM solver. The model was trained using a combination of Mean Squared Error (MSE) loss and an energy residual loss derived from the variational equation. The dataset consisted of 2D traction vectors as the input labels and their corresponding displacement fields as the target applied to the problem shown in Figure 1. To evaluate the effectiveness of the U-Net in predicting the RKPM solution, the performance was compared against simple polynomial regression models. An evaluation framework consisting of qualitative and quantitative metrics was assessed.

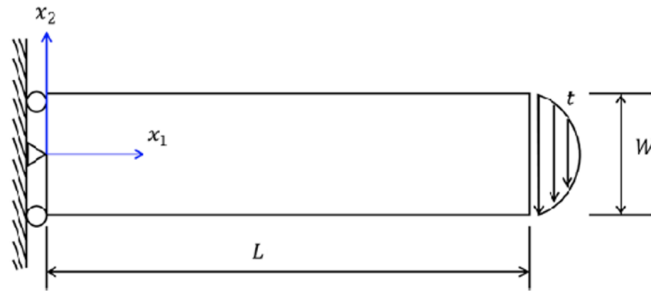


Figure 1: Problem setting of 2D cantilever beam under an example distributed shear load.

2. Dataset

The dataset was generated using a previously developed RKPM solver script written in MATLAB. The displacements were obtained using a linear basis, cubic B-spline shape functions, and equally spaced nodes. Different combinations of load types were considered, such as distributed loads and point loads, with the magnitudes of the load parameters varying by case. 10 load types were used as a basis: vertical/horizontal point load, uniformly distributed horizontal/shear load, parabolically distributed horizontal/shear load, linearly increasing horizontal/shear load, and linearly decreasing horizontal/shear load. To increase the size and variability of the load cases, additional samples were created by scaling the base cases by 0.5 and 1.5. The principle of superposition was leveraged to create combinations of base cases and scaled cases. In total, 1830 samples were produced using this method. All load cases were applied to a standardized 2D simply-supported cantilever beam with the dimensions labelled in Figure 1. The dimensions of the beam used were length $L = 48$ and width $W = 20$. The beam was assigned a Young's modulus of 1 and Poisson's ratio of 0.3. For analysis, the domain was discretized into 49×21 nodes for an average nodal spacing of 1. Furthermore, the free end was designated as the natural boundary at which all loads were applied. The essential boundary was denoted by the simple supports on the left-hand side and kept consistent across all load cases.

After running each load case, the resulting displacements were separated into their x- (u_1) and y- (u_2) components. The solutions for each case were stored as a 1029×2 matrix where the rows correspond to the displacements at each node, and the columns correspond to the dimensions. Additionally, the tractions for each case were saved as a matrix with the same dimensions. In this study, the displacements were the target, while the tractions were the labels. The essential boundary conditions in the nodal space were saved as a standalone mesh grid matrix to be used as an input. This acted as an initialization of the nodal displacements, where the values are the prescribed displacements on the essential boundary and zero elsewhere. The node-based input provided a more meaningful feature representation than a latent noise input used by traditional convolution-based models. Similarly, the material constitutive $\mathbf{D}$ matrix for plane strain was saved to be used in energy residual loss calculation. The RKPM shape functions for each node and their gradients were included in the dataset but were not used in training the U-Net.

To process the dataset, the two-column matrices in each sample were reshaped into the original 49×21 mesh grid. The samples were then shuffled, and a standard 80%/20% split was adopted for the training and testing datasets. The data were scaled using the global absolute maximum of the training set displacements and tractions separately. This bounded all dataset samples between $[-1, 1]$. Example displacement field pair and deformed shape plots for a single sample subjected to horizontal linearly decreasing traction are shown in Figures 2 and 3, respectively.

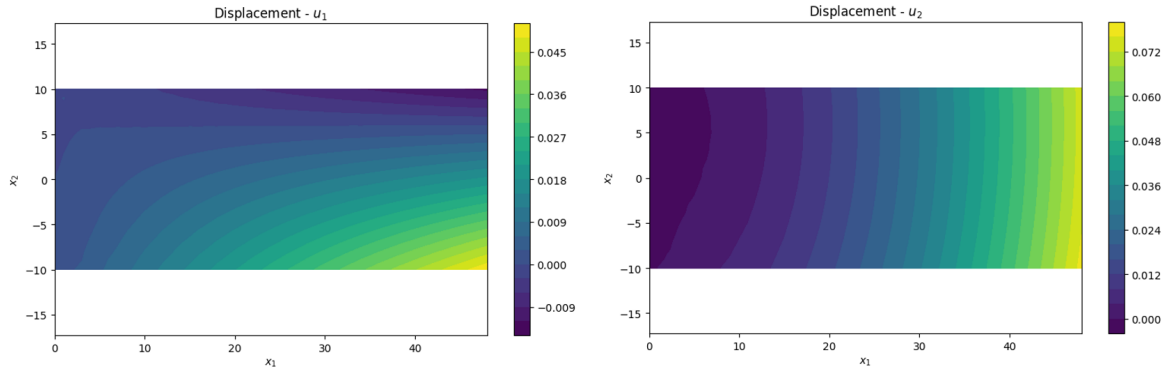


Figure 2: Sample displacement fields in x and y.

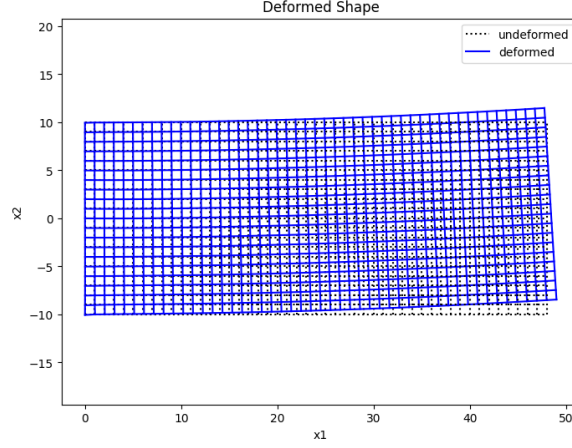


Figure 3: Sample deformed shape.

3. Methodology

3.1. Overall Framework

The U-Net is trained using a concatenation of the prescribed nodal displacements and nodal traction field. A schematic of the training procedure is detailed in Figure 4. For this U-Net, each pixel value corresponds to a positive or negative displacement, depending on the direction channel. This differs from vanilla U-Nets, where each channel represents a color and each pixel value represents a color or grayscale intensity. Thus, the predicted displacement field for a prescribed set of displacements given prescribed tractions is produced by the U-Net instead of a segmented image.

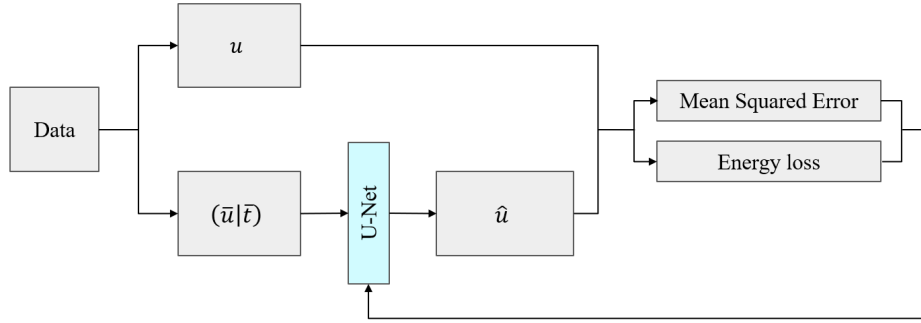


Figure 4: U-Net training process featuring physics-based loss function.

The training framework incorporates a commonly used mean squared error (MSE) loss used in regression-style problems, in addition to a physics-based energy residual loss. Traditional Physics-Informed Neural Networks PINN use a strong-form (residual) loss for solving PDE problems [9, 10]. In this study, the variational equation with Niche’s method for imposing boundary conditions was used as a basis to formulate an energy-based residual loss.

3.2. Model Architecture

The architecture of the U-Net is presented in Figure 5. Output dimensions for each layer are indicated where relevant. The U-Net takes the Dirichlet and Neumann boundary conditions discretized in nodal space as inputs. The boundary conditions are separated into their spatial components to create a four-channel input. The resulting output is the two-channel displacement field in both x- and y- components.

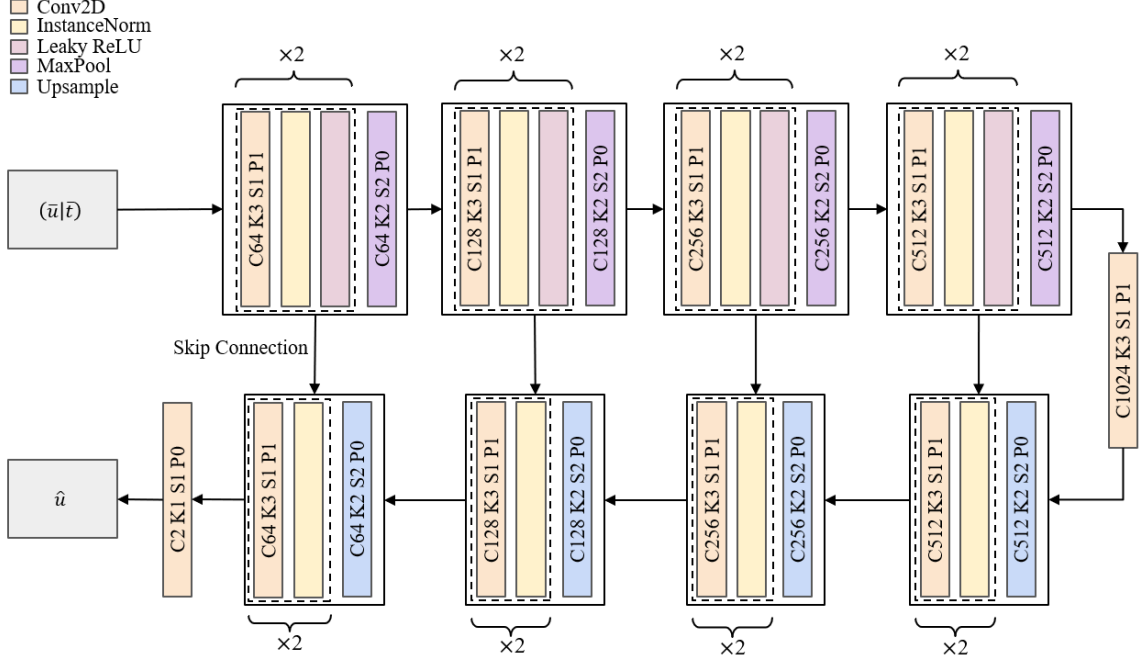


Figure 5: Architecture of the U-Net used for displacement field generation. C represents channels, K represents kernel size, S represents stride, and P represents padding.

Overall, the network follows the conventional symmetric structure consisting of an encoder and decoder, which gives it the characteristic U-shaped layout. Furthermore, the model retains the skip connections used in the original U-Net [5]. Skip connections are used to help the model remember key features during downsampling. Some modifications were made to the traditional architecture, such as the addition of instance normalization layers, which help improve the generalization ability of the network [11]. The ReLU activation functions were replaced by LeakyReLU layers with a slope of 0.1 to help the model predict negative displacements. Likewise, the transposed convolution layers were replaced with bilinear interpolation layers for upsampling, since the spatial dimensions of the input were irregular. The activation functions were completely removed in the convolution blocks of the decoder pipeline to avoid excessively clipping the predicted displacements.

3.3. Training Procedure

The U-Net was trained using a combination of a standard MSE loss and energy-based physical loss function.

3.3.1. MSE Loss

The MSE loss directly compares predicted values with corresponding target values. This loss helps to constrain the magnitude of the predicted displacements and is defined as

$$\mathcal{L}_{MSE} = \frac{1}{m} \sum_i^m (\mathbf{u}^{(i)} - \hat{\mathbf{u}}^{(i)})^2 \quad (1)$$

where m represents the number of samples in a batch and i is the index of the sample. True displacements are given by $\mathbf{u}$ while predicted displacements are denoted by $\hat{\mathbf{u}}$.

3.3.2. Energy Residual Loss

Physics-aware losses help to guide the network towards plausible predictions by incorporating physical relations in the data [7–9]. In this study, the total potential energy is considered, and Niche’s method is used to enforce boundary conditions. This formulation is consistent with the RKPM approximation used to obtain the dataset. Energy can be expressed as a variational equation in the form

$$\Pi = \frac{1}{2} \int_{\Omega} \sigma_{ij} \epsilon_{ij} d\Omega - \int_{\Omega} u_i b_i d\Omega - \int_{\Gamma_t} u_i \bar{t}_i d\Gamma - \int_{\Gamma_u} \sigma_{ij} n_j (u_i - \bar{u}_i) d\Gamma + \frac{\beta}{2} \int_{\Gamma_u} (u_i - \bar{u}_i)^2 d\Gamma \quad (2)$$

where σ_{ij} represents the components of the Cauchy stress tensor, $\epsilon_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i})$ is the strain, b_i denotes the body force per unit area, $\bar{t}_i$ is the prescribed traction on the Neumann boundary Γ_t , $\bar{u}_i$ is the prescribed displacement on the Dirichlet boundary Γ_u , and β is a penalty coefficient enforcing the Dirichlet boundary condition. The penalty parameter is defined using

$$\beta = \frac{100E}{h} \quad (3)$$

where E is the material Young's modulus and h is the average nodal spacing. The stress was calculated using

$$\sigma_{ij} = \mathbf{D}\epsilon_{ij} \quad (4)$$

Equation 2 can be simplified since body force is not applied in the problem statement. Thus, the second term can be ignored. The loss function aims to minimize the absolute relative error between the true total energy and the total energy of the predicted solution. Each of the integrals can be numerically evaluated by discretizing the domain and boundary using the finite element mesh. Letting k index the quadrature points, the discretized energy as a function of generic displacements u^* can be expressed as

$$\begin{aligned} \Pi^h(u^*) = & \left| \frac{1}{2} \sum_{q=1}^{N_\Omega} \sigma_{ij}^{(m)}(x_q) \epsilon_{ij}^{(m)}(x_q) w_q^\Omega - \sum_{\substack{q=1 \\ x_q \in \Gamma_t}}^{N_\Gamma} u_i^{*(m)}(x_q) \bar{t}_i^{(m)}(x_q) w_q^\Gamma \right. \\ & \left. - \sum_{\substack{q=1 \\ x_q \in \Gamma_u}}^{N_\Gamma} \sigma_{ij}^{(m)}(x_q) n_j(x_q) (u_i^{*(m)}(x_q) - \bar{u}_i^{(m)}(x_q)) w_q^\Gamma + \frac{\beta}{2} \sum_{\substack{q=1 \\ x_q \in \Gamma_u}}^{N_\Gamma} (u_i^{*(m)}(x_q) - \bar{u}_i^{(m)}(x_q))^2 w_q^\Gamma \right| \end{aligned} \quad (5)$$

where x_q is the quadrature point, w_p is the quadrature weight, N_Ω is the number of quadrature points in the domain Ω , and N_Γ is the number of quadrature points on the boundary Γ . The summation over $q = 1, \dots, N_\Omega$ and $q = 1, \dots, N_\Gamma$ evaluates the integrals using these points. Equation 5 can be evaluated for both true and predicted displacements to obtain the energy residual loss

$$\mathcal{L}_{\text{Energy}} = \frac{1}{m} \sum_i^m \frac{|\Pi^h(\hat{\mathbf{u}}^{(i)}) - \Pi^h(\mathbf{u}^{(i)})|}{\Pi^h(\mathbf{u}^{(i)})} \quad (6)$$

This allows the physics-based residual loss to be consolidated into a single term instead of several residual terms like in conventional PINN.

3.3.3. U-Net Loss

The total loss used to update the U-Net can be defined as the linear combination of the MSE and the energy residual losses.

$$\mathcal{L}_{\text{UNet}} = \lambda_{\text{MSE}} \mathcal{L}_{\text{MSE}} + \lambda_{\text{Energy}} \mathcal{L}_{\text{Energy}} \quad (7)$$

where each λ is the weight of the loss functions used, denoted by their subscript.

3.3.4. Training Strategy and Hyperparameters

The network was trained using the Adam optimizer with a momentum parameter of 0.9. Batches of size 256 were used during training over 200 epochs. The learning rate of the U-Net was initialized at 5×10^{-3} and set to decay by 20% when validation loss plateaued for more than 10 epochs. The relative weights for each loss component are detailed in Table 1.

Table 1: Weights assigned to each loss during training. MSE loss weight was scaled to match the magnitude of the energy residual loss.

Weight	Value
λ_{MSE}	1
λ_{Energy}	1×10^{-3}

Each of the hyperparameters was systematically tuned by sweeping across a range of values based on those discussed in existing literature. The results of each test were evaluated based on the visual and numerical

results of the testing set. When selecting loss weights, both the training and validation histories for each loss component were used.

Training was conducted on a T4 GPU using Google Colaboratory.

3.3.5. Comparison Models

Traditional machine learning methods were based on regression techniques [12]. Regression models in the simplest form aim to solve for the weights θ in the following polynomial

$$\hat{\mathbf{u}} = \theta^T \mathbf{x} \quad (8)$$

where $\mathbf{x}$ is the input features. The polynomial degree of the regression model depends on the polynomial degree of the features. Increasing the degree can increase the complexity and variance but also risks leading to overfitting. Therefore, the polynomial degree is generally fine-tuned when training a regression model. To assess the effectiveness of the U-Net, the network performance is compared with the results obtained by training both a linear and a cubic regression model trained using the Sci-Kit Learn library. The regression models adopted the ridge regression training strategy, which introduces a penalty-based regularization term into the loss function

$$\mathcal{L} = \sum_i^m (u^{(i)} - \hat{u}^{(i)})^2 + \alpha \|\theta\|^2 \quad (9)$$

where α is the regularization term. To identify the best value for regularization, a 5-fold cross-validation was performed using code borrowed from instructional lab sessions. The regularization parameters for each polynomial model are summarized in Table 2.

Table 2: Regularization parameters determined using cross-validation.

Model	Penalty
Linear	1×10^{-5}
Cubic	1×10^{-4}

Since the high-dimensional nodal fields are unsuitable as inputs for the regression models, each sample was condensed into eight scalar features. These features consist of the mean and standard deviation of the prescribed displacements and applied tractions in each spatial direction assembled as a single vector for each sample.

4. Results and Evaluation

The U-Net was assessed using a combination of qualitative and quantitative comparisons against the performance of the polynomial regression models.

4.0.1. Qualitative Visual Comparison

The displacements can be visually evaluated by plotting the deformed shape predicted by the network superimposed on the undeformed shape. A comparison of the model predictions against the target displacements are plotted for three randomly selected samples from the test dataset in Figures 6-8. The plotted deformations were magnified by a factor of 20 for visual clarity.

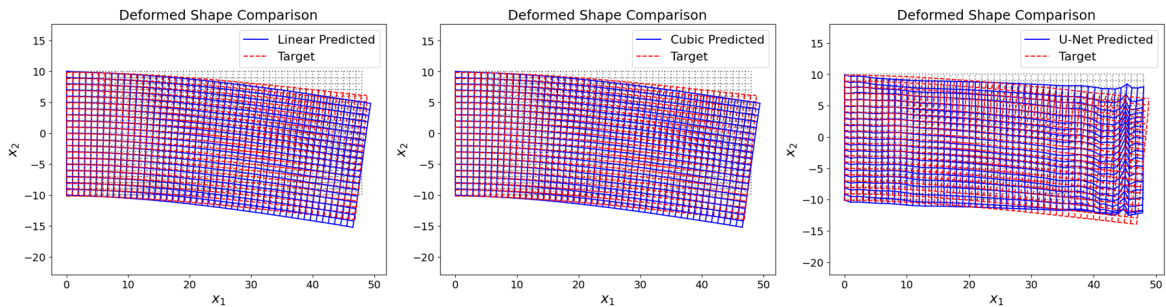


Figure 6: Deformation comparison of a sample with uniformly distributed negative shear.

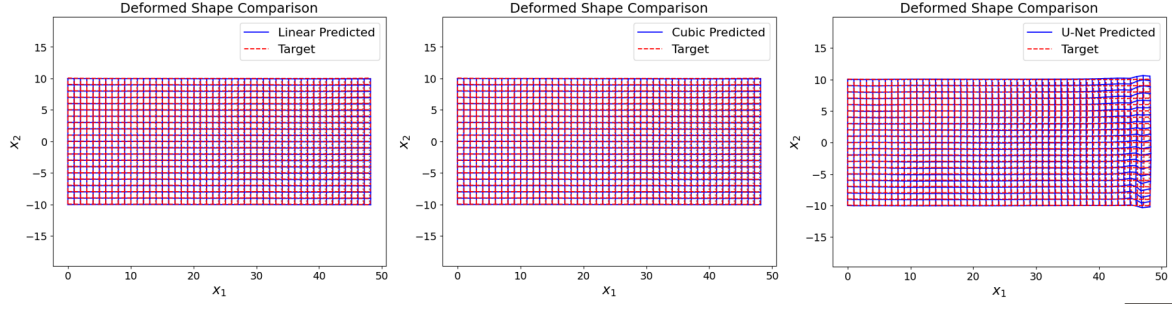


Figure 7: Deformation comparison of a sample with uniformly distributed horizontal tension and linearly decreasing distributed shear that transitions from positive to negative.

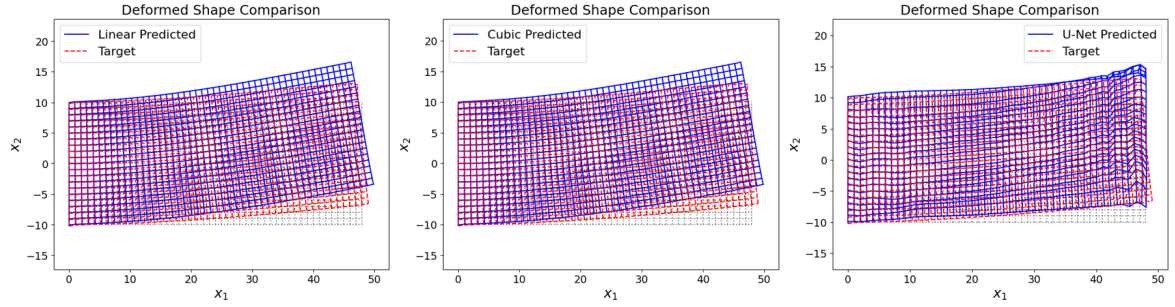


Figure 8: Deformation comparison of a sample with linearly increasing distributed positive shear.

Additionally, the displacement field for each sample can be further examined by plotting the distribution of each component. The contour plots for the predicted displacement fields are plotted alongside the true displacement fields and their absolute error in Figures 9-11. The plots are provided as the u_1 component followed by the u_2 component for each model.

Both polynomial models were able to capture the direction of the deformation. However, they noticeably overestimate vertical deformations during bending. This suggests that the simple regression models learn to interpolate the trends in displacements, which is supported by the fact that they are trained on statistical information instead of the full field inputs. Conversely, the U-Net performs better from a visual evaluation. Unlike the regression models, the U-Net overestimates deformations significantly less. However, it should be noted that the predictions tend to have warping, primarily around the Neumann boundary. This is likely due to the energy residual loss guiding the model predictions towards correct boundary conditions.

The contour plots show that the U-Net can mostly reproduce the displacement distributions in the ground truth. As observed in the deformed shapes, there are minor distortions in the displacement fields predicted by the U-Net compared to those of the regression models. Regardless, when analyzing the error plots, the U-Net displays considerably less error magnitude, with only small jumps where distortions occur. This further suggests that the physics-aware training contributed to prediction accuracy.

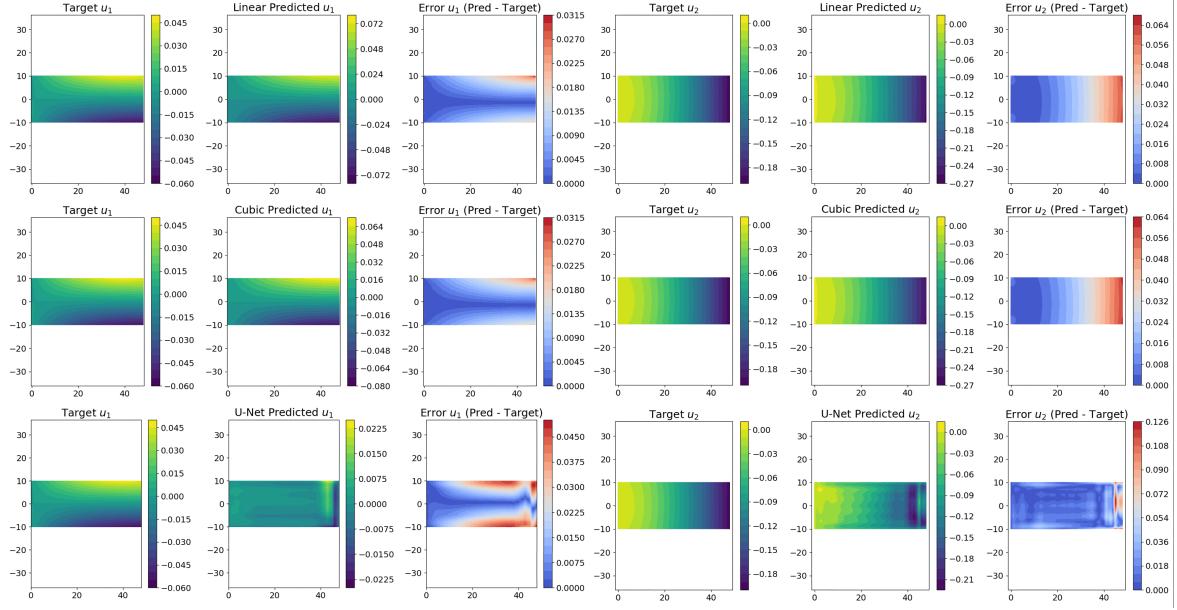


Figure 9: Contour plot comparison of a sample with uniformly distributed negative shear.

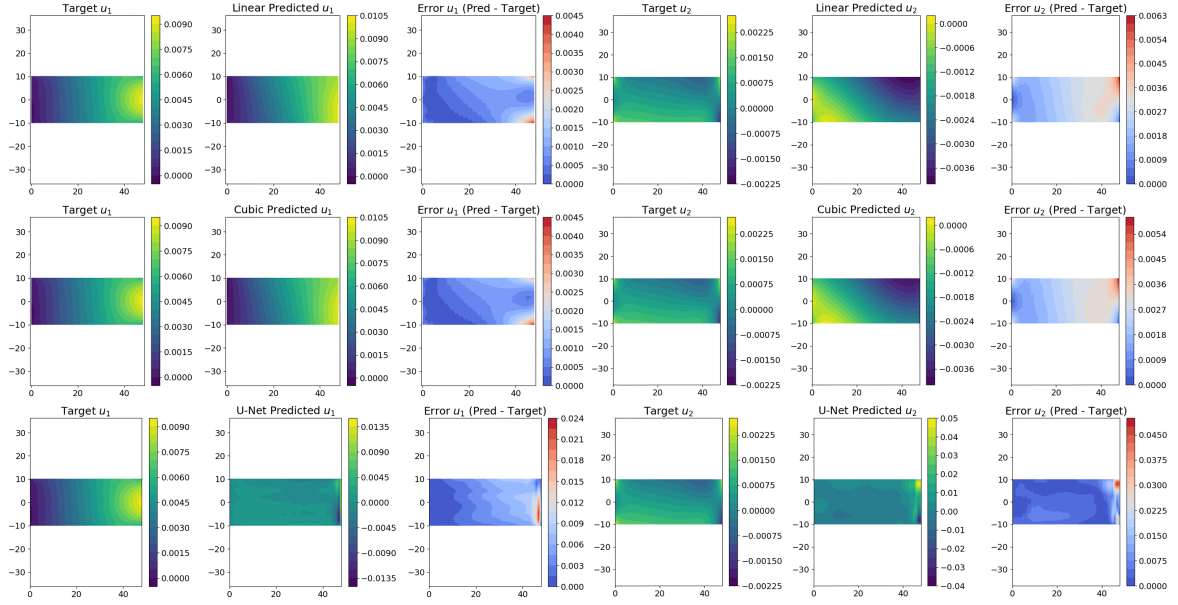


Figure 10: Contour plot comparison of a sample with uniformly distributed horizontal tension and linearly decreasing distributed shear that transitions from positive to negative.

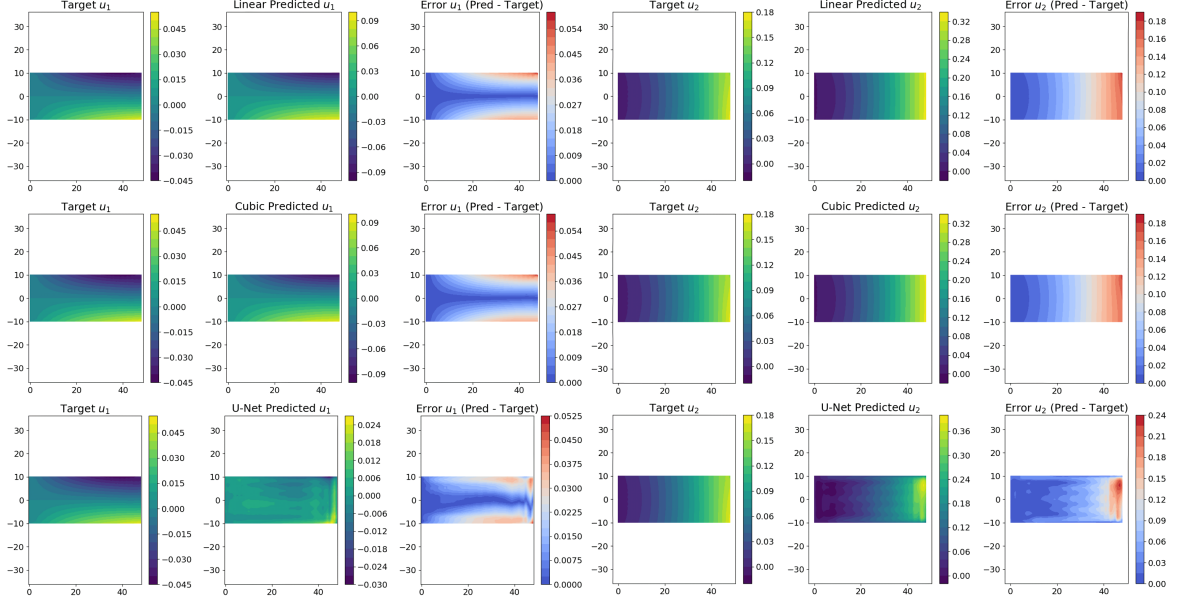


Figure 11: Contour plot comparison of a sample with linearly increasing distributed positive shear.

4.0.2. Quantitative Loss Comparison

The performance of each model can be quantitatively assessed by applying the MSE and energy residual losses on the model predictions for the test dataset. A summary of testing losses is detailed in Table 3.

From the testing losses, the two regression models perform similarly, with the cubic regression model only slightly improving over the linear model. The similarity between the polynomial models suggests that their performance was limited by the lack of spatial context in the input features. In contrast, the U-Net greatly outperformed the regression models, scoring one order of magnitude lower in both metrics. This supports the effectiveness of the physical loss in guiding the model towards predicting realistic displacements, which overall improved the results compared to a purely MSE-driven approach.

Table 3: Testing MSE and energy residual losses for each model.

Model	MSE	Energy
Linear Regression	0.000830	3.277600
Cubic Regression	0.000828	3.265300
U-Net	0.000038	0.365661

5. Conclusions

This study investigated a physics-informed approach to training a U-Net for a 2D elasticity problem. The energy-based residual loss, coupled with the spatial learning of the U-Net, was able to predict realistic displacement fields for a set of nodal prescribed boundary conditions given the traction field.

Overall, the U-Net is shown to be able to capture both the trend of the deformation and the magnitude of displacement, consistent with the energy of the ground truth solution. This advantage pushes the model ahead of traditional regression models, which fail to predict the correct magnitudes of the displacements. While the U-Net performs better in replicating the deformed shape, one notable flaw is that there is some warping present around the boundaries. One possibility is due to the balancing of the loss components, as the magnitude of the total energy for each sample can be significantly larger than the MSE, which posed a challenge during training. Another potential limitation of the approach is the architecture of the model. While skip connections help the model to remember features from the input, they could also be causing the distortions observed in the predicted solutions. The training protocol could be improved in the future by trying different loss formulations or loss weight decay to further improve balancing. Another change that could be made is to modify the architecture, such as removing or replacing layer types.

6. Appendix

All Jupyter Notebooks used for training, as well as the dataset and auxiliary training scripts, can be accessed on GitHub: https://github.com/DoanJ7313/Physics_Informed_U-Net or on Google Drive: https://drive.google.com/drive/folders/1U5bF-VX-6mN9UPSLozbwqIWhyRJYIvuD?usp=drive_link.

References

- [1] R. W. Clough, Original formulation of the finite element method, *Finite elements in analysis and design* 7 (2) (1990) 89–101.
- [2] W. K. Liu, S. Jun, Y. F. Zhang, Reproducing kernel particle methods, *International journal for numerical methods in fluids* 20 (8-9) (1995) 1081–1106.
- [3] W. K. Liu, Y. Chen, S. Jun, J. Chen, T. Belytschko, C. Pan, R. Uras, C. Chang, Overview and applications of the reproducing kernel particle methods, *Archives of Computational Methods in Engineering* 3 (1) (1996) 3–80.
- [4] T. Belytschko, J.-S. Chen, M. Hillman, *Meshfree and particle methods: fundamentals and applications*, John Wiley & Sons, 2023.
- [5] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: *International Conference on Medical image computing and computer-assisted intervention*, Springer, 2015, pp. 234–241.
- [6] W. Jiangtao, N. I. R. Ruhaiyem, F. Panpan, A comprehensive review of u-net and its variants: Advances and applications in medical image segmentation, *IET Image Processing* 19 (1) (2025) e70019.
- [7] J. Baek, J.-S. Chen, K. Susuki, A neural network-enhanced reproducing kernel particle method for modeling strain localization, *International Journal for Numerical Methods in Engineering* 123 (18) (2022) 4422–4454.
- [8] Y. Wang, J.-S. Chen, S. E. Casebolt, Neural network-enriched rkpm for dynamics based on action minimization, *Computer Methods in Applied Mechanics and Engineering* 451 (2026) 118662.
- [9] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations, *arXiv preprint arXiv:1711.10561* (2017).
- [10] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, F. Piccialli, Scientific machine learning through physics-informed neural networks: Where we are and what’s next, *Journal of Scientific Computing* 92 (3) (2022) 88.
- [11] X.-Y. Zhou, G.-Z. Yang, Normalization in training u-net for 2-d biomedical semantic segmentation, *IEEE Robotics and Automation Letters* 4 (2) (2019) 1792–1799.
- [12] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, <http://www.deeplearningbook.org>.